```
int strtype::operator==(strtype s2)
{
    return strcmp(s, s2.s) == 0;
}

int main()
{
    strtype o1("Hello"), o2("There"), o3;

    o3 = o1 + o2;
    cout << o3.get() << '\n';

    o3 = o1;
    if(o1==o3)
        cout << "o1 equals o3\n";

    if(o1>o2)
        cout << "o1 > o2\n";

    if(o1<o2)
        cout << "o1 < o2\n";\

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 7

1. No. Overloading an operator simply expands the data types upon which it can operate, but no preexisting operations are affected.

2. Yes. You cannot overload an operator relative to one of C++'s built-in types.

3. No, the precedence cannot be changed. No, the number of operands cannot be altered.

4.
```
#include <iostream>
using namespace std;

class array
{
    int nums[10];
public:
    array();
    void set(int n[10]);
    void show();
    array operator+(array ob2);
    array operator-(array ob2);
    int operator==(array ob2);
};

array::array()
{
    int i;
```

```
    for(i=0; i<10; i++)
        nums[i] = 0;
}

void array::set(int *n)
{
    int i;
    for(i=0; i<10; i++)
        nums[i] = n[i];
}

void array::show()
{
    int i;

    for(i=0; i<10; i++)
        cout << nums[i] << ' ';

    cout << "\n";
}

array array::operator+(array ob2)
{
    int i;
    array temp;

    for(i=0; i<10; i++)
        temp.nums[i] = nums[i] + ob2.nums[i];

    return temp;
}

array array::operator-(array ob2)
{
    int i;
    array temp;

    for(i=0; i<10; i++)
        temp.nums[i] = nums[i] - ob2.nums[i];

    return temp;
}

int array::operator==(array ob2)
{
    int i;

    for(i=0; i<10; i++)
        if(nums[i]!=ob2.nums[i])
        return 0;
```

```
    return 1;
}

int main()
{
    array o1, o2, o3;

    int i[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    o1.set(i);
    o2.set(i);

    o3 = o1 + o2;
    o3.show();

    o3 = o1 - o3;
    o3.show();

    if(o1==o2)
        cout << "o1 equals o2\n";
    else
        cout << "o1 does not equal o2\n";

    if(o1==o3)
        cout << "o1 equals o3\n";
    else
        cout << "o1 does not equal o3\n";

    return 0;
}
```

5.
```
#include <iostream>
using namespace std;

class array
{
    int nums[10];
public:
    array();
    void set(int n[10]);
    void show();
    friend array operator+(array ob1, array ob2);
    friend array operator-(array ob1, array ob2);
    friend int operator==(array ob1, array ob2);
};

array::array()
{
    int i;
    for(i=0; i<10; i++)
        nums[i] = 0;
```

```
}

void array::set(int *n)
{
    int i;
    for(i=0; i<10; i++)
        nums[i] = n[i];
}

void array::show()
{
    int i;

    for(i=0; i<10; i++)
        cout << nums[i] << ' ';

    cout << "\n";
}

array operator+(array ob1, array ob2)
{
    int i;
    array temp;

    for(i=0; i<10; i++)
        temp.nums[i] = ob1.nums[i] + ob2.nums[i];

    return temp;
}

array operator-(array ob1, array ob2)
{
    int i;
    array temp;

    for(i=0; i<10; i++)
        temp.nums[i] = ob1.nums[i] - ob2.nums[i];

    return temp;
}

int operator==(array ob1, array ob2)
{
    int i;

    for(i=0; i<10; i++)
        if(ob1.nums[i]!=ob2.nums[i])
        return 0;

    return 1;
}
```

```
int main()
{
    array o1, o2, o3;

    int i[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    o1.set(i);
    o2.set(i);

    o3 = o1 + o2;
    o3.show();

    o3 = o1 - o3;
    o3.show();

    if(o1==o2)
        cout << "o1 equals o2\n";
    else
        cout << "o1 does not equal o2\n";

    if(o1==o3)
        cout << "o1 equals o3\n";
    else
        cout << "o1 does not equal o3\n";

    return 0;
}
```

6.
```
#include <iostream>
using namespace std;

class array
{
    int nums[10];
public:
    array();
    void set(int n[10]);
    void show();
    array operator++();
    friend array operator--(array &ob);
};

array::array()
{
    int i;
    for(i=0; i<10; i++)
        nums[i] = 0;
}

void array::set(int *n)
{
```

```
    int i;
    for(i=0; i<10; i++)
        nums[i] = n[i];
}

void array::show()
{
    int i;

    for(i=0; i<10; i++)
        cout << nums[i] << ' ';

    cout << "\n";
}

// Overload unary op using member function.
array array::operator++()
{
    int i;

    for(i=0; i<10; i++)
        nums[i]++;

    return *this;
}

// Use a friend./
array operator--(array &ob)
{
    int i;

    for(i=0; i<10; i++)
        ob.nums[i]--;

    return ob;
}


int main()
{
    array o1, o2, o3;

    int i[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    o1.set(i);
    o2.set(i);

    o3 = ++o1;
    o3.show();

    o3 = --o1;
```

```
    o3.show();

    return 0;
}
```

7. No. To overload the assignment operator you must use a member function.

7.1 EXERCISES

1. A and C are legal statements.

2. A public member of a base becomes a public member of a derived class when it is inherited as public. When a public member of a base is inherited as private, it becomes a private member of the derived class.

7.2 EXERCISES

1. When a protected member of a base class is inherited as public, it becomes a protected member of the derived class. If it is inherited as private, it becomes a private member of the derived class. If it is inherited as protected, it becomes a protected member of the derived class.

2. The protected category is needed to allow a base class to keep certain members private while still allowing a derived class to have access to them.

3. No.

7.3 EXERCISES

1.
```
#include <iostream>
#include <cstring>
using namespace std;

class mybase
{
    char str[80];
public:
    mybase(char *s) { strcpy(str, s); }
    char *get() { return str; }
};

class myderived : public mybase
{
    int len;
public:
    myderived(char *s) : mybase(s)
    {
        len = strlen(s);
    }
    int getlen() { return len; }
    void show() { cout << get() << '\n'; }
```

```
};

int main()
{
    myderived ob("hello");

    ob.show();
    cout << ob.getlen() << '\n';

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

// A base class for various types of vehicles.
class vehicle
{
    int num_wheels;
    int range;
public:
    vehicle(int w, int r)
    {
        num_wheels = w;
        range = r;
    }
    void showv()
    {
        cout << "Wheels: " << num_wheels << '\n';
        cout << "Range: " << range << '\n';
    }
};

class car : public vehicle
{
    int passengers;
public:
    car(int p, int w, int r) : vehicle(w, r)
    {
        passengers = p;
    }
    void show()
    {
        showv();
        cout << "Passengers: " << passengers << '\n';
    }
};

class truck : public vehicle
{
    int loadlimit;
```

```
public:
    truck(int l, int w, int r) : vehicle(w, r)
    {
        loadlimit = l;
    }
    void show()
    {
        showv();
        cout << "loadlimit " << loadlimit << '\n';
    }
};

int main()
{
    car c(5, 4, 500);
    truck t(30000, 12, 1200);

    cout << "Car: \n";
    c.show();
    cout << "\nTruck:\n";
    t.show();

    return 0;
}
```

7.4 EXERCISES

1. ```
   Constructing A
   ```
   ```
   Constructing B
   ```
   ```
   Constructing C
   ```
   ```
   Destructing C
   ```
   ```
   Destructing B
   ```
   ```
   Destructing A
   ```

2. ```
   #include <iostream>
   using namespace std;

   class A
   {
       int i;
   public:
       A(int a) { i = a; }
   };

   class B
   {
       int j;
   public:
   ```

```
    B(int a) { j = a; }
};

class C : public A, public B
{
    int k;
public:
    C(int c, int b, int a) : A(a), B(b)
    {
        k = c;
    }
};
```

7.5 EXERCISES

2. A virtual base class is needed when a derived class inherits two (or more) classes, both of which are derived from the same base class. Without virtual base classes, two (or more) copies of the common base class would exist in the final derived class, However, if the original base is virtual, only one copy is present in the final derived class.

MASTERY SKILLS CHECK: Chapter 7

1. ```
   #include <iostream>
   using namespace std;

   class building
   {
   protected:
       int floors;
       int rooms;
       double footage;
   };

   class house : public building
   {
       int bedrooms;
       int bathrooms;
   public:
       house(int f, int r, double ft, int br, int bth)
       {
           floors = f;
           rooms = r;
           footage = ft;
           bedrooms = br;
           bathrooms = bth;
       }
       void show()
       {
           cout << "floors: " << floors << '\n';
   ```
   ```

```
        cout << "rooms: " << rooms << '\n';
        cout << "square footage: " << footage << '\n';
        cout << "bedrooms: " << bedrooms << '\n';
        cout << "bathrooms: " << bathrooms << '\n';
    }
};

class office : public building
{
    int phones;
    int extinguishers;
public:
    office(int f, int r, double ft, int p, int ext)
    {
        floors = f;
        rooms = r;
        footage = ft;
        phones = p;
        extinguishers = ext;
    }
    void show()
    {
        cout << "floors: " << floors << '\n';
        cout << "floors: " << floors << '\n';
        cout << "rooms: " << rooms << '\n';
        cout << "square footage: " << footage << '\n';
        cout << "Telephones: " << phones << '\n';
        cout << "fore extinguishers: ";
        cout << extinguishers << '\n';
    }
};

int main()
{
    house h_ob(2, 12, 5000, 6, 4);
    office o_ob(4, 25, 12000, 30, 8);

    cout << "House:\n";
    h_ob.show();

    cout << "\nOffice:\n";
    o_ob.show();

    return 0;
}
```

2. When a base class is inherited as public, the public members of the base become public members of the derived class, and the base's private members remain private to the base. If the base is inherited as private, all members of the base become private members of the derived class.

3. Members declared as **protected** are private to the base class but can be inherited (and

accessed) by any derived class. When used as an inheritance access specifier, **protected** causes all public and protected members of the base class to become protected members of the derived class.

4. Constructors are called in order of derivation. Destructors are called in reverse order.

5.
```
#include <iostream>
using namespace std;

class planet
{
protected:
    double distance; // miles from the sun
    int revolve; // in days
public:
    planet(double d, int r) { distance = d; revolve = r; }
};

class earth : public planet
{
    double circumference;
public:
    earth(double d, int r) : planet(d, r)
    {
        circumference = 2*distance*3.1416;
    }
    void show()
    {
        cout << "Distance from sun: " << distance << '\n';
        cout << "Days in orbit: " << revolve << '\n';
        cout << "Circumference of orbit: ";
        cout << circumference << '\n';
    }
};

int main()
{
    earth ob(93000000, 365);

    ob.show();

    return 0;
}
```

6. To fix the program, have **motorized** and **road_use** inherit **vehicle** as a virtual base class. Also, refer to Question 1 in the Cumulative Skills Check in this chapter.

CUMULATIVE SKILLS CHECK: Chapter 7

1. Some compilers will not allow you to use a **switch** in an in-line function. If this is the case with your compiler, the functions were automatically made into "regular" functions.

2. The assignment operators is the only operator that is not inherited. The reason for this is easy to understand. Since a derived class will contain members not found in the base class, the overloaded = relative to the base has no knowledge of the members added by the derived class and, as such, cannot properly copy those new members.

REVIEW SKILLS CHECK: Chapter 8

1.
```
#include <iostream>
using namespace std;

class airship
{
protected:
    int passengers;
    double cargo;
};

class airplane : public airship
{
    char engine; // p for propeller, j for jet
    double range;
public:
    airplane(int p, double c, char e, double r)
    {
        passengers = p;
        cargo = c;
        engine = e;
        range = r;
    }
    void show();
};

class balloon : public airship
{
    char gas; // h for hydrogen, e for helium
    double altitude;
public:
    balloon(int p, double c, char g, double a)
    {
        passengers = p;
        cargo = c;
        gas = g;
        altitude = a;
    }
    void show();
};

void airplane::show()
{
    cout << "Passengers: " << passengers << '\n';
    cout << "Cargo capacity: " << cargo << '\n';
```